

Tham lam (Greedy Algorithms)

PHỤ THUỘC A/04 cho lập luận đối chỗ — chương này không dùng được nếu thiếu nó; B/07 cho heap; B/11 cho DSU trong Kruskal.

TÓM TẮT

Tham lam là chiến lược có khoảng cách lớn nhất giữa độ ngắn của code và độ khó của chứng minh: bốn dòng chương trình, nhưng nếu không chứng minh thì bạn không biết mình đúng hay sai. Chương này tập trung vào phần chứng minh — hai khuôn chuẩn là lập luận đối chỗ và “tham lam luôn đi trước” — và vào câu hỏi sâu hơn: *cấu trúc nào của bài toán làm cho tham lam đúng?* Câu trả lời có tên riêng là matroid, và biết nó giúp bạn nhận ra ngay một lớp bài mà tham lam chắc chắn tối ưu. Chương cũng nói về những nơi tham lam sai nhưng vẫn hữu ích: xấp xỉ với bảo đảm chất lượng. Nếu chỉ nhớ một điều: tham lam không sai vì “nhìn ngắn”, nó sai vì lựa chọn hôm nay *phá huỷ* khả năng của tương lai — và nhiệm vụ của chứng minh là loại trừ đúng khả năng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thuật toán tham lam** — xây lời giải bằng cách lặp lại một lựa chọn tốt nhất theo tiêu chí cục bộ, không bao giờ xét lại.
- **tính chất lựa chọn tham lam** — tồn tại một lời giải tối ưu chứa lựa chọn mà tham lam đưa ra ở bước đầu. Đây là điều phải chứng minh.
- **lập luận đối chỗ** — biến một lời giải tối ưu bất kỳ thành lời giải tham lam qua các phép đổi không làm nó tệ đi (A/04).
- **matroid** — cấu trúc gồm một tập nền và một họ “tập độc lập” thỏa hai tính chất; đó chính xác là cấu trúc mà tham lam luôn cho kết quả tối ưu.
- **hàm dưới mô-đun (submodular)** — hàm mà lợi ích biên giảm dần khi tập đã chọn lớn lên. Tham lam trên hàm này cho bảo đảm xấp xỉ $1 - 1/e$.
- **tham lam có hối tiếc** — biến thể: cứ nhận tất cả, rồi khi vi phạm ràng buộc thì bỏ đi phần tử tệ nhất đã nhận — thường cài bằng heap.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao tham lam sai ở bài đổi tiền với mệnh giá $\{1, 3, 4\}$ mà lại đúng với mệnh giá tiền thật của hầu hết các nước?
- Q2. Kruskal và Prim đều là tham lam, đều cho cây khung nhỏ nhất, nhưng chọn cạnh theo hai tiêu chí hoàn toàn khác nhau. Vì sao cả hai cùng đúng?
- Q3. Có cách nào biết *trước* rằng tham lam sẽ đúng cho một bài, thay vì thử rồi hy vọng?
- Q4. Khi tham lam sai, vì sao câu trả lời thường là quy hoạch động chứ không phải một tiêu chí tham lam khôn hơn?
- Q5. Thuật toán tham lam cho bài phủ tập hợp không tối ưu nhưng vẫn được dùng rộng rãi. Điều gì làm nó đáng tin cậy dù không tối ưu?

Có một trải nghiệm phổ biến khi luyện thuật toán: bạn đọc đề, nghĩ ra một quy tắc đơn giản — “cứ chọn cái rẻ nhất”, “cứ chọn cái kết thúc sớm nhất” — cài trong ba phút, nộp, và sai. Bạn thử

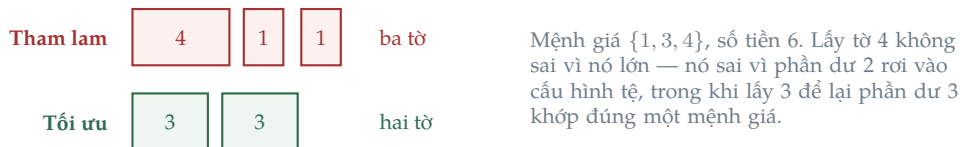
một quy tắc khác, cũng sai. Đến quy tắc thứ ba thì đúng, nhưng bạn không biết vì sao, và bài sau bạn lại phải đoán từ đầu.

Đó là trạng thái mà chương này muốn chấm dứt. Tham lam không phải trò may rủi; nó có lý thuyết, và lý thuyết đó trả lời được câu “bài này tham lam có đúng không” trước khi bạn gõ code. Nhưng phải nói ngay điều khó chịu: cái giá của việc dùng tham lam là bạn *phải chứng minh*, và không có cách nào né. Một lời giải tham lam không có chứng minh chỉ là một phỏng đoán được viết bằng ngôn ngữ lập trình.

1.1 Vì sao tham lam sai, nói cho chính xác

Trực giác thông thường nói tham lam sai vì nó “nhìn ngắn”. Cách nói đó đúng nhưng quá mơ hồ để dùng. Phát biểu chính xác hơn: tham lam sai khi lựa chọn hôm nay *phá huỷ những khả năng* mà một lựa chọn khác giữ lại được.

Hãy xem bài đổi tiền với mệnh giá $\{1, 3, 4\}$ và số tiền 6. Tham lam lấy 4, còn lại 2, phải lấy $1 + 1$ — ba tờ. Tối ưu là $3 + 3$ — hai tờ. Vấn đề không phải ở chỗ 4 lớn hơn 3, mà ở chỗ sau khi lấy 4 thì phần dư 2 rơi vào một cấu hình tệ, trong khi lấy 3 để lại phần dư 3 rơi đúng vào mệnh giá có sẵn.



Hình 1.1. Vì sao tham lam sai, nói cho chính xác. Vấn đề không nằm ở chỗ lựa chọn hiện tại “nhỏ” hay “lớn” mà ở chỗ nó *phá huỷ* cấu hình còn lại. Mọi chứng minh trong chương này đều nhằm loại trừ đúng khả năng đó.

Từ cách phát biểu này rút ra ngay một cách kiểm tra thực dụng: *sau khi tham lam chọn một phần tử, bài toán còn lại có còn cùng dạng và có tệ đi so với phương án khác không?* Nếu bạn chứng minh được rằng chọn theo tiêu chí tham lam để lại một bài toán con “không tệ hơn” mọi lựa chọn khác, thì tham lam đúng — và đó chính là nội dung của lập luận đổi chỗ.

1.2 Hai khuôn chứng minh

Chương A/04 đã giới thiệu hai khuôn; ở đây ta dùng chúng trên các bài thật.

Khuôn 1 — đổi chỗ. Lấy một lời giải tối ưu bất kỳ, xét chỗ khác đầu tiên so với tham lam, chứng minh rằng có thể sửa lời giải tối ưu tại đó cho giống tham lam mà không làm nó tệ đi. Áp dụng cho bài xếp lịch: tham lam chọn cuộc họp kết thúc sớm nhất; nếu lời giải tối ưu chọn cuộc khác thì đổi sang cuộc kết thúc sớm nhất vẫn hợp lệ vì nó kết thúc sớm hơn, nên không cản trở các cuộc sau.

Khuôn 2 — tham lam luôn đi trước. Chứng minh bằng quy nạp rằng sau k bước, lời giải tham lam “không kém” bất kỳ lời giải nào theo một đại lượng đo được. Với bài xếp lịch, đại lượng là thời điểm kết thúc của cuộc họp thứ k : tham lam luôn có thời điểm kết thúc sớm nhất có thể, nên nó không bao giờ bị kẹt trước lời giải khác.

Trong thực hành, khuôn 1 thường dễ áp hơn vì nó chỉ đòi hỏi một phép biến đổi cục bộ. Nhưng khuôn 2 có ưu điểm riêng: khi phép đổi chỗ không thực hiện được, chỗ vướng thường chính là phần ví dụ đang cần tìm.

Có một mẹo phụ rất hữu ích cho các bài mà tiêu chí tham lam là “sắp xếp theo cái gì đó”: nếu bạn nghi ngờ nên sắp theo tiêu chí f , hãy xét *hai phần tử liền kề* trong một lời giải tối ưu và hỏi “đổi chỗ hai phần tử này có làm kết quả tốt lên không”. Điều kiện để không đổi chỗ được sẽ cho bạn chính xác tiêu chí sắp xếp. Đây là cách người ta suy ra các tiêu chí lạ như “sắp theo tỉ số” hay “sắp theo hiệu” mà nhìn đề thì không đoán ra.

1.3 Matroid: cấu trúc mà tham lam luôn đúng

Đến đây có một câu hỏi tự nhiên: thay vì chứng minh lại cho từng bài, có đặc trưng chung nào cho các bài mà tham lam đúng không? Có, và nó là một trong những kết quả đẹp nhất của lý thuyết thuật toán.

Một *matroid* gồm một tập nền E và một họ các tập con được gọi là “độc lập”, thoả hai điều kiện: mọi tập con của một tập độc lập cũng độc lập; và nếu tập độc lập A nhỏ hơn tập độc lập B thì luôn tồn tại một phần tử trong $B \setminus A$ thêm được vào A mà vẫn giữ tính độc lập. Điều kiện thứ hai — gọi là tính chất trao đổi — chính là thứ đảm bảo rằng tham lam không bao giờ tự đưa mình vào ngõ cụt.

Định lý cốt lõi: *thuật toán tham lam cho kết quả tối ưu trên mọi hàm trọng số khi và chỉ khi cấu trúc là một matroid*. Chiều thuận nói tham lam đúng; chiều đảo còn mạnh hơn — nếu cấu trúc không phải matroid thì tồn tại một hàm trọng số làm tham lam sai.

Ví dụ điển hình: tập các tập cạnh không tạo chu trình của một đồ thị tạo thành một matroid. Đó là lý do Kruskal đúng — và cũng là câu trả lời cho câu hỏi mở đầu thứ hai: Kruskal và Prim khác nhau ở tiêu chí nhưng cùng khai thác một cấu trúc, nên cả hai cùng đúng. Một ví dụ khác: bài xếp lịch các công việc có hạn chót, mỗi việc một đơn vị thời gian, tối đa hoá lợi nhuận — tập các việc “xếp được” cũng tạo thành matroid.

Giá trị thực hành của khái niệm này không phải để bạn kiểm tra tiên đề mỗi lần gặp bài, mà là để có một danh mục: nếu bài của bạn quy về “chọn tập con độc lập trọng số lớn nhất” trên một cấu trúc kiểu rừng, kiểu không gian vector, hay kiểu phân hoạch, thì tham lam gần như chắc chắn đúng và bạn không cần đắn đo.

TRỰC GIÁC

Vì sao tính chất trao đổi lại là điều kiện đúng? Hãy nghĩ theo hướng ngược lại. Tham lam thất bại khi nó chọn xong k phần tử và rơi vào tình trạng “không thêm được gì nữa”, trong khi một lời giải khác chọn k phần tử khác lại còn mở rộng được. Tính chất trao đổi loại trừ đúng tình huống này: nếu lời giải kia lớn hơn thì luôn có phần tử của nó thêm được vào tập của tham lam. Nói cách khác, matroid là cách phát biểu hình thức cho điều kiện “tham lam không bao giờ tự bịt đường của chính mình”.

1.4 Danh mục các bài tham lam kinh điển

Bảng dưới đây gom các bài mà tiêu chí tham lam đã được chứng minh, kèm tiêu chí và lý do. Đáng thuộc vì các bài mới thường là biến thể của chúng.

Bài toán	Tiêu chí tham lam	Vì sao đúng
Chọn nhiều cuộc họp nhất	Kết thúc sớm nhất trước	Để lại nhiều thời gian nhất cho phần còn lại; đổi chỗ trực tiếp
Cây khung nhỏ nhất	Cạnh nhẹ nhất không tạo chu trình (Kruskal)	Cấu trúc rừng là matroid; tính chất cắt
Cây khung nhỏ nhất	Cạnh nhẹ nhất rời khỏi cây đang có (Prim)	Cùng tính chất cắt, duyệt theo thứ tự khác
Ba lô phân số	Tỉ số giá trị trên khối lượng	Được lấy phần lẻ nên không có “lãng phí chỗ”
Mã Huffman	Gộp hai tần suất nhỏ nhất	Hai ký hiệu hiếm nhất phải nằm sâu nhất; đổi chỗ
Xếp lịch giảm trễ tối đa	Hạn chót sớm nhất trước	Đổi chỗ hai việc liên kế không làm trễ tăng
Đổi tiền (hệ chuẩn)	Mệnh giá lớn nhất trước	Chỉ đúng với hệ mệnh giá đặc biệt — phải kiểm
Chọn việc có hạn chót, tối đa lợi nhuận	Lợi nhuận lớn trước, xếp muộn nhất có thể	Matroid phân hoạch theo thời điểm
Phủ tập hợp	Chọn tập phủ nhiều phần tử mới nhất	<i>Không</i> tối ưu, nhưng bảo đảm xấp xỉ $\ln n$

Có một mẫu đáng chú ý trong bảng: gần như mọi tiêu chí đều là một *thứ tự sắp xếp*. Điều đó nổi chương này với chương B/06: rất nhiều bài tham lam thực chất là “sắp xếp theo đúng khoá rồi quét một lượt”, và toàn bộ độ khó nằm ở việc tìm ra khoá.

Tham lam có hối tiếc

Một biến thể đáng thuộc vì nó giải một họ bài mà tham lam thuần tuý chịu. Ý tưởng: cứ nhận mọi thứ đến; khi vi phạm ràng buộc thì bỏ đi phần tử tệ nhất trong những cái đã nhận. Cấu trúc thực thi là heap (B/07).

Ví dụ: mỗi công việc có hạn chót và thời lượng, cần chọn nhiều việc nhất làm được. Duyệt theo hạn chót tăng dần, nhận mọi việc; nếu tổng thời gian vượt hạn chót hiện tại thì bỏ việc dài nhất đã nhận. Kết quả tối ưu, và chứng minh vẫn là đổi chỗ.

Điều đáng học ở đây có tính phương pháp: khi tiêu chí tham lam “chọn hay không chọn” quá khó, hãy đổi sang tiêu chí “nhận hết rồi loại dần”. Hai cách nhìn cho cùng bài toán, nhưng cách thứ hai thường có tiêu chí đơn giản hơn nhiều.

1.5 Khi tham lam sai: xấp xỉ có bảo đảm

Không phải cứ tham lam sai là vứt đi. Với nhiều bài NP-khó (D/23), tham lam là thuật toán tốt nhất trong thực tế, và điều làm nó đáng tin không phải là nó “thường đúng” mà là nó có *bảo đảm chất lượng chứng minh được*.

Hai kết quả kinh điển. Với bài phủ tập hợp, tham lam chọn tập phủ được nhiều phần tử mới nhất cho lời giải không tệ hơn $\ln n$ lần tối ưu — và người ta chứng minh được rằng dưới các giả thuyết chuẩn của lý thuyết độ phức tạp thì không thuật toán đa thức nào làm tốt hơn đáng kể. Nghĩa là tham lam ở đây không phải giải pháp tạm mà là gần như tốt nhất có thể.

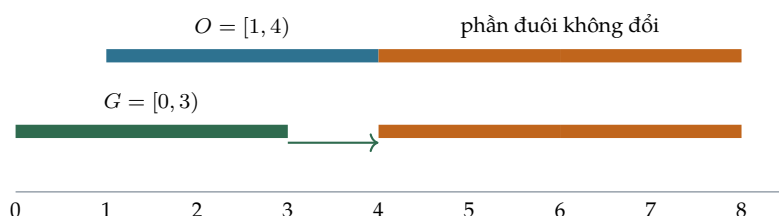
Kết quả thứ hai đẹp hơn và có ứng dụng rộng: khi hàm mục tiêu là *dưới mô-đun* — lợi ích biên giảm dần khi tập đã chọn lớn lên — và ta tối đa hoá dưới ràng buộc số lượng, thì tham lam cho ít nhất $1 - 1/e \approx 63\%$ giá trị tối ưu. Tính chất lợi ích biên giảm dần xuất hiện tự nhiên ở rất nhiều nơi: chọn vị trí đặt cảm biến, chọn tập ảnh đại diện, chọn đặc trưng cho mô hình, chọn khung hình chủ chốt trong một hệ thống bản đồ. Trong tất cả những bài đó, tham lam vừa đơn giản vừa có bảo đảm — một sự kết hợp hiếm.

CẠM BẮY

Bảo đảm xấp xỉ là bảo đảm về *trường hợp xấu nhất*, không phải mô tả hành vi thường gặp. Tham lam cho phủ tập hợp trong thực tế thường cho kết quả rất gần tối ưu, tốt hơn nhiều so với cận $\ln n$. Ngược lại, đừng lấy hành vi thực tế tốt làm cơ sở để bỏ qua việc kiểm tra: có những dữ liệu chạm đúng trường hợp xấu, và trong thi đấu thì dữ liệu *được cố ý* dựng như vậy.

1.6 Vẽ phép đổi chỗ: điều gì được bảo toàn

Trong bài chọn nhiều cuộc họp nhất, dùng đoạn nửa mở $[s, f)$ để hai cuộc chạm đầu mút vẫn tương thích. Hình 1.2 giữ nguyên phần đuôi của một lịch tối ưu, chỉ thay cuộc đầu tiên. Lý do đúng là $f(G) \leq f(O)$, không phải vì độ dài cuộc G ngắn hơn.



Hình 1.2. Đổi cuộc đầu tiên của lịch tối ưu sang cuộc kết thúc sớm nhất không làm mất cuộc nào ở phía sau. Đoạn trống từ 3 tới 4 cho thấy phần thời gian được nới ra sau phép đổi.

Nếu mỗi cuộc có lợi ích khác nhau, phép đổi vẫn giữ tính hợp lệ nhưng có thể giảm tổng lợi ích. Ví dụ $[0, 3)$ lợi ích 2 và $[0, 4)$ lợi ích 100: chọn kết thúc sớm cho giá trị 2 thay vì 100. Khi đó dùng quy hoạch động trên các cuộc đã sắp theo thời điểm kết thúc, với trạng thái có lựa chọn lấy hoặc bỏ cuộc hiện tại. Kinh nghiệm rút ra: mỗi lần đề thêm trọng số, hãy kiểm lại đại lượng được bảo toàn trong chứng minh cũ.

1.7 Gốc rễ: điện khí hoá Moravia và một bài tập về nhà

Bài toán cây khung nhỏ nhất có một nguồn gốc bất ngờ và rất cụ thể. Năm 1926, nhà toán học Séc Otakar Borůvka công bố thuật toán đầu tiên cho bài toán này, và động cơ là một nhu cầu kỹ thuật của thời đó: quy hoạch mạng lưới điện cho vùng Moravia sao cho tổng chiều dài dây dẫn nhỏ nhất. Thuật toán của ông — gộp đồng thời mọi thành phần với cạnh nhẹ nhất rời khỏi chúng — về sau được đánh giá cao vì rất dễ song song hoá, dù suốt nhiều thập kỷ nó ít nổi tiếng hơn hai thuật toán sau. Jarník tìm ra thuật toán kiểu Prim năm 1930, cũng ở Tiệp Khắc; Kruskal và Prim công bố độc lập vào năm 1956 và 1957 tại Mỹ, trong bối cảnh nghiên cứu mạng viễn thông.

Mã Huffman có một câu chuyện thường được kể trong giới: năm 1952, David Huffman là sinh viên cao học ở MIT, và giáo sư cho lớp chọn giữa việc thi cuối kỳ hoặc viết một tiểu luận về việc tìm mã tiền tố tối ưu — một bài toán mà chính giáo sư và các đồng nghiệp chưa giải được. Huffman chọn tiểu luận, gần như bỏ cuộc, rồi tìm ra ý tưởng gộp từ dưới lên. Điều đáng học không phải giai thoại mà là hướng suy nghĩ: mọi người trước đó cố xây cây *từ trên xuống*, còn lời giải nằm ở hướng ngược lại. Đây là một ví dụ trực tiếp cho heuristic “đảo ngược hướng nhìn” ở chương A/01.

Phần lý thuyết đến muộn hơn nhiều và từ một ngành khác. Khái niệm matroid do Hassler Whitney đưa ra năm 1935 trong bối cảnh toán học thuần túy — ông muốn trừu tượng hoá khái niệm

“độc lập tuyến tính” để nó dùng được cho cả đồ thị. Mãi tới đầu thập niên 1970, Jack Edmonds mới nối hai đầu lại và chứng minh rằng matroid *chính xác* là cấu trúc mà tham lam tối ưu. Đây là một mẫu lặp lại trong lịch sử ngành: một khái niệm toán học ra đời không vì ứng dụng, ba mươi năm sau trở thành câu trả lời chính xác cho một câu hỏi thuật toán.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1926 — Borůvka	Quy hoạch mạng điện cho vùng Moravia với tổng chiều dài dây nhỏ nhất.	Thuật toán cây khung nhỏ nhất đầu tiên; ít nổi tiếng nhưng song song hoá tốt nhất trong ba thuật toán.
1930, 1956–57 — Jarník, Kruskal, Prim	Thiết kế mạng viễn thông chi phí thấp.	Hai tiêu chí tham lam khác nhau cho cùng bài toán; cả hai đều đúng vì cùng khai thác tính chất cắt.
1952 — Huffman	Tìm mã tiền tố tối ưu — bài toán mở mà thầy giao thay cho bài thi.	Xây cây từ dưới lên thay vì từ trên xuống; ví dụ trực tiếp của heuristic “đảo ngược hướng nhìn”.
1935 — Whitney	Trừu tượng hoá khái niệm độc lập tuyến tính cho cả đồ thị.	Matroid ra đời trong toán thuần túy, không nhằm ứng dụng nào.
Đầu thập niên 1970 — Edmonds	Vì sao tham lam đúng ở bài này mà sai ở bài kia?	Định lý: tham lam tối ưu trên mọi trọng số khi và chỉ khi cấu trúc là matroid — một điều kiện <i>cần và đủ</i> , hiếm gặp trong lý thuyết thuật toán.
1978 — Nemhauser và cộng sự	Tối đa hoá hàm có lợi ích biên giảm dần dưới ràng buộc số lượng.	Tham lam bảo đảm $1 - 1/e$; nền lý thuyết cho việc chọn cảm biến, chọn đặc trưng, chọn dữ liệu đại diện.

I 1.8 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kinh tế học	Quyết định cận biên, tối ưu hoá thiên cận	Cùng cấu trúc: chọn theo lợi ích tức thời. Cùng chỗ hỏng: khi lựa chọn hôm nay thay đổi tập lựa chọn ngày mai. Kinh tế học gọi đó là chi phí cơ hội động; ta gọi là “phá huỷ khả năng tương lai”.
Tiến hoá, tối ưu hoá cục bộ	Leo đồi: luôn đi theo hướng cải thiện	Tham lam là leo đồi trên không gian tổ hợp. Cùng thất bại: kẹt ở cực trị địa phương. Cách chữa cũng tương tự — cho phép bước lùi, tức là mô phỏng luyện kim (C/19).
Hệ điều hành, bộ nhớ đệm	Chính sách thay thế LRU, lập lịch tiến trình	LRU là tham lam trên thông tin quá khứ. Điểm khác quan trọng: ở đây tương lai <i>chưa biết</i> , nên tiêu chuẩn đánh giá không phải tối ưu mà là tỉ lệ cạnh tranh so với lời giải biết trước tương lai (D/24).
Nén dữ liệu	Mã Huffman, mã số học	Ứng dụng tham lam kinh điển nhất, và vẫn đang chạy trong mọi tệp ảnh nén bạn mở hôm nay.
Học máy	Chọn đặc trưng tham lam, cây quyết định, học tích cực	Cây quyết định chọn thuộc tính chia tốt nhất tại mỗi nút — tham lam thuần túy, và nó <i>không</i> tối ưu toàn cục nhưng chạy tốt trong thực tế. Nhiều tiêu chí chọn dữ liệu có tính dưới mô-đun nên tham lam có bảo đảm $1 - 1/e$.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Robot và SLAM	Chọn khung hình chủ chốt, chọn điểm mốc, chọn cảm biến	Bài toán “chọn k quan sát cho nhiều thông tin nhất” thường có hàm mục tiêu dưới mô-đun — lợi ích của một quan sát mới giảm khi đã có nhiều quan sát tương tự. Đó là lý do tham lam ở đây không chỉ tiện mà còn có bảo đảm chất lượng.
Tài chính	Chiến lược thiên cận so với tối ưu động	Cùng bài học: tối ưu từng bước không cho tối ưu cả chuỗi khi có ràng buộc liên thời gian. Đây cũng chính là ranh giới dẫn sang quy hoạch động ở C/16.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì tính đúng của tham lam ở bài đổi tiền là tính chất của *hệ mệnh giá*, không phải của thuật toán. Với $\{1, 3, 4\}$ và số tiền 6, lấy 4 để lại phần dư 2 rơi vào cấu hình tệ, trong khi lấy 3 để lại phần dư 3 khớp đúng một mệnh giá. Các hệ tiền thật thường có cấu trúc bội số $(1, 2, 5, 10, \dots)$ làm cho việc lấy mệnh giá lớn nhất không bao giờ phá hủy khả năng của phần dư. Có thuật toán kiểm tra một hệ mệnh giá có “tham lam được” hay không, và đó là bằng chứng rõ ràng rằng đây là tính chất của dữ liệu chứ không của phương pháp.
- A2. Vì cả hai đều dựa trên cùng một tính chất, gọi là tính chất cắt: với mọi cách chia đỉnh thành hai nhóm, cạnh nhẹ nhất nối hai nhóm luôn thuộc một cây khung nhỏ nhất nào đó. Kruskal áp dụng nó bằng cách duyệt cạnh theo trọng số tăng; Prim áp dụng bằng cách luôn xét cắt giữa cây đang có và phần còn lại. Nói theo ngôn ngữ mục 3: tập các tập cạnh không tạo chu trình là một matroid, và mọi cách duyệt tham lam hợp lệ trên matroid đều cho kết quả tối ưu.
- A3. Có, ít nhất cho một lớp lớn: nếu bài toán quy về “chọn tập con độc lập có trọng số lớn nhất” trên một cấu trúc thoả hai tiên đề matroid thì tham lam chắc chắn tối ưu, và định lý của Edmonds nói điều ngược lại cũng đúng — không phải matroid thì tồn tại trọng số làm tham lam sai. Trong thực hành, thay vì kiểm tiên đề, người ta nhận diện qua danh mục: cấu trúc rừng, không gian vector, phân hoạch theo nhóm. Ngoài lớp đó thì vẫn phải chứng minh bằng đổi chỗ cho từng bài.
- A4. Vì khi không có lựa chọn cục bộ nào an toàn, vấn đề không nằm ở việc tiêu chí chưa đủ khôn mà nằm ở chỗ *lời giải tối ưu phụ thuộc vào những gì đã chọn trước đó*. Nói cách khác, bài toán có trạng thái. Quy hoạch động là cách xử lý đúng cho tình huống đó: thay vì cam kết một lựa chọn, ta xét mọi lựa chọn nhưng nhớ lại kết quả theo trạng thái để khởi tính lại (C/16). Cố tìm tiêu chí tham lam khôn hơn trong tình huống này thường dẫn tới một tiêu chí phức tạp mà vẫn có phản ví dụ.
- A5. Bảo đảm chất lượng *chứng minh được*: nó không bao giờ tệ hơn tối ưu quá $\ln n$ lần cho bài phủ tập hợp, hoặc không kém $1 - 1/e$ lần tối ưu khi hàm mục tiêu có lợi ích biên giảm dần. Điều này khác hẳn với “thực nghiệm thấy tốt”: bảo đảm đúng cho mọi dữ liệu, kể cả dữ liệu do đối thủ dựng. Và với bài phủ tập hợp còn có kết quả mạnh hơn: dưới các giả thuyết chuẩn về độ phức tạp, không thuật toán đa thức nào cải thiện được cận đó đáng kể — nên tham lam gần như là tốt nhất có thể.

NÓI LẠI THẬT ĐƠN GIẢN

Tham lam là cách giải mà ai cũng thử đầu tiên: cứ chọn cái tốt nhất trước mắt, rồi tính tiếp. Nhiều khi nó đúng, nhiều khi nó sai, và chương này nói về cách phân biệt hai trường hợp đó. Hãy nghĩ tới việc trả tiền lẻ. Nếu có các tờ 1, 2, 5, 10 thì cứ lấy tờ to nhất còn dùng được là ổn. Nhưng nếu tiền chỉ có mệnh giá 1, 3 và 4, mà bạn phải trả 6, thì lấy tờ 4 trước là hỏng — còn 2 và bạn phải dùng hai tờ 1, tổng ba tờ; trong khi hai tờ 3 là đủ. Điều làm hỏng không phải vì tờ 4 nhỏ hay to, mà vì sau khi lấy nó thì phần còn lại rơi vào tình thế xấu. Đó chính là câu hỏi duy nhất cần hỏi mỗi lần định dùng tham lam: *sau khi tôi chọn cái này, phần còn lại có bị kẹt hơn so với chọn cái khác không?* Nếu chứng minh được là không bao giờ kẹt hơn, tham lam đúng. Cách chứng minh chuẩn cũng rất trực quan: đưa tôi bất kỳ cách làm tốt nhất nào của bạn, tôi sẽ sửa dần nó cho giống cách của tôi, mỗi lần sửa đều không làm nó xấu đi.

Có một điều thú vị mà các nhà toán học đã tìm ra: tồn tại một loại cấu trúc mà trên đó tham lam *luôn luôn* đúng, bất kể trọng số là gì. Nếu bài toán của bạn có dạng “chọn một nhóm phần tử sao cho không tạo thành vòng” — như chọn dây điện nối các thành phố mà không tạo vòng lặp — thì bạn ở trong loại cấu trúc đó và có thể yên tâm.

Cuối cùng, khi tham lam sai, đôi khi ta vẫn dùng nó — nhưng chỉ khi biết rõ nó sai tới mức nào. Có những bài mà tham lam bảo đảm cho kết quả ít nhất bằng khoảng hai phần ba mức tốt nhất. Biết con số đó là điều khác biệt giữa “dùng liều” và “dùng có căn cứ”.

Một câu: Tham lam không sai vì nhìn ngắn; nó sai khi lựa chọn hôm nay bịt mắt đường của ngày mai.

Chỗ không rút gọn được: Tìm ra tiêu chí tham lam đúng cho một bài mới vẫn là bước sáng tạo; matroid chỉ nhận diện được một lớp, không phải mọi bài.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Tham lam sai khi nào — phát biểu chính xác	Khi lựa chọn hiện tại phá hủy những khả năng mà lựa chọn khác giữ lại được, chứ không phải vì “nhìn ngắn”.
●●● Hai khuôn chứng minh	Đổi chỗ (biến lời giải tối ưu thành tham lam, từng bước không tệ đi) và “tham lam luôn đi trước” (quy nạp theo một đại lượng đo được).
●●● Mẹo tìm tiêu chí sắp xếp	Xét hai phần tử liền kề trong lời giải tối ưu, hỏi “đổi chỗ có tốt lên không”; điều kiện không đổi được cho ra chính tiêu chí sắp xếp.
●●● Matroid — định lý cốt lõi	Tham lam tối ưu trên mọi hàm trọng số <i>khi và chỉ khi</i> cấu trúc là matroid. Rừng trong đồ thị là matroid, nên Kruskal đúng.
●●● Tính chất cắt	Với mọi cách chia đỉnh thành hai nhóm, cạnh nhẹ nhất nối hai nhóm thuộc một cây khung nhỏ nhất — nền của cả Kruskal lẫn Prim.
●●● Khi tham lam sai thì làm gì	Chuyển sang quy hoạch động, vì bài toán có trạng thái: lời giải tối ưu phụ thuộc những gì đã chọn trước đó ($C/16$).
●● Tham lam có hối tiếc	Nhận hết, khi vi phạm thì bỏ phần tử tệ nhất đã nhận (dùng heap). Đổi tiêu chí “chọn hay không” thành “nhận rồi loại”, thường đơn giản hơn nhiều.
●● Huffman	Gộp hai tần suất nhỏ nhất; ý tưởng đến từ việc xây cây <i>từ dưới lên</i> thay vì từ trên xuống.
●● Ba lô phân số so với ba lô nguyên	Phân số: tham lam theo tỉ số, tối ưu. Nguyên: tham lam sai, cần quy hoạch động — khác nhau đúng ở chỗ “có lấy phần lẻ được không”.
●● Bảo đảm xấp xỉ của tham lam	Phủ tập hợp: không tệ hơn $\ln n$ lần tối ưu. Hàm dưới mô-đun dưới ràng buộc số lượng: ít nhất $1 - 1/e$ lần tối ưu.
●● Hàm dưới mô-đun	Lợi ích biên giảm dần khi tập đã chọn lớn lên; xuất hiện tự nhiên ở chọn cảm biến, chọn đặc trưng, chọn khung hình chủ chốt.
● Bảo đảm xấp xỉ nghĩa là gì	Bảo đảm cho trường hợp xấu nhất, không mô tả hành vi thường gặp — thực tế thường tốt hơn nhiều, nhưng dữ liệu thi đấu được dựng để chạm đúng trường hợp xấu.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Trong kỳ thi, tôi nên dành bao lâu để chứng minh một tiêu chí tham lam?

| Một quy trình thực dụng: dành hai phút thử đổi chỗ trong đầu; nếu ra thì cài. Nếu không ra sau hai phút, hãy viết bộ sinh test ngẫu nhiên nhỏ và so với vết cặn — năm phút và cho câu trả lời

chắc chắn hơn nhiều so với ngồi nghĩ thêm mười phút. Nếu stress test không tìm được phản ví dụ sau vài nghìn ca nhỏ, xác suất tiêu chí đúng là khá cao. Điều tuyệt đối không nên làm là cài rồi nộp mà không làm cả hai việc trên.

2. Bài của tôi có nhiều tiêu chí tham lam hợp lý. Chọn thế nào?

| Dùng mẹo hai phần tử liền kề ở mục 2: giả sử lời giải tối ưu xếp phần tử a ngay trước b ; viết ra điều kiện để việc đổi chỗ chúng *không* cải thiện kết quả. Bất đẳng thức thu được chính là tiêu chí so sánh cần dùng để sắp xếp, và nó thường có dạng bất ngờ như “sắp theo $a_i + b_i$ ” hoặc “sắp theo tỉ số”. Cách này biến việc đoán tiêu chí thành việc giải một bất đẳng thức — xác định hơn nhiều.

3. Nếu tham lam cho kết quả gần đúng thì có nên nộp không?

| Trong thi đấu thì không, vì chấm là đúng hoặc sai. Trong công việc thật thì tùy, và câu hỏi đúng là: bạn có *bảo đảm* nào không? “Thử vài ca thấy gần đúng” rất khác với “chứng minh được không tệ hơn $1 - 1/e$ lần tối ưu”. Nếu có bảo đảm và sai số chấp nhận được thì tham lam là lựa chọn tốt: đơn giản, nhanh, dễ bảo trì. Nếu không có bảo đảm, ít nhất hãy đo trên dữ liệu thật và biết phân phối sai số.

4. Tham lam và quy hoạch động có thể kết hợp không?

| Có, theo nhiều cách. Cách phổ biến nhất: dùng tham lam để giảm không gian trạng thái — chứng minh rằng một số lựa chọn không bao giờ tối ưu và loại chúng, rồi quy hoạch động trên phần còn lại. Cách thứ hai: dùng lời giải tham lam làm cận trên trong nhánh cận (C/13). Cách thứ ba, thường gặp trong bài khó: sắp xếp bằng tiêu chí tham lam trước, rồi quy hoạch động trên thứ tự đó — vì tiêu chí đúng thu hẹp bài toán từ “mọi thứ tự” xuống “một thứ tự”.

5. Vì sao Dijkstra được xếp vào tham lam?

| Vì nó cam kết một lựa chọn không bao giờ xét lại: mỗi bước chốt đỉnh có nhãn nhỏ nhất và coi khoảng cách đó là cuối cùng. Chứng minh tính đúng chính là chứng minh lựa chọn tham lam an toàn, và nó dùng giả thiết trọng số không âm đúng một lần — ở chỗ khẳng định rằng không có đường nào đi vòng qua các đỉnh chưa chốt rồi quay lại mà ngắn hơn. Đây là ví dụ tốt cho việc “giả thiết nằm ở đâu trong chứng minh” quyết định “thuật toán hỏng ở đâu”.

6. Tính dưới mô-đun kiểm tra thế nào trong thực tế?

| Định nghĩa là: thêm một phần tử vào tập nhỏ cho lợi ích ít nhất bằng thêm chính phần tử đó vào tập lớn hơn chứa nó. Trong thực hành, ba dấu hiệu gợi ý mạnh: hàm đếm số phần tử được phủ (phủ tập hợp); hàm đo lượng thông tin thu được từ các quan sát có phần trùng lặp; và hàm đo độ đa dạng của một tập đã chọn. Ba họ này phủ phần lớn ứng dụng thực tế. Nếu nghi ngờ, hãy kiểm tra bằng số trên các trường hợp nhỏ trước khi dựa vào bảo đảm lý thuyết.

7. Có bài nào mà tham lam đúng nhưng chứng minh rất khó không?

| Có, và một ví dụ nổi tiếng là bài xếp lịch với hạn chót và tiền phạt, hay các biến thể của bài toán ghép đôi ổn định. Thuật toán ghép đôi ổn định — mỗi bên lần lượt cầu hôn theo thứ tự ưu tiên — trông rất giống tham lam và luôn cho kết quả ổn định, nhưng chứng minh đòi hỏi lập luận tinh tế về việc không tồn tại cặp muốn bỏ nhau. Bài học: độ ngắn của code không nói gì về độ khó của chứng minh, và đó là chủ đề của cả chương này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Chọn nhiều cuộc họp nhất không chồng lấn	CSES “Movie Festival”[1]	Sắp theo thời điểm kết thúc; viết chứng minh đổi chỗ ra giấy.
● Ba lô phân số	Bài kinh điển	Sắp theo tỉ số; giải thích vì sao cách này sai với ba lô nguyên.
● Số tiền nhỏ nhất không tạo được từ các mệnh giá đã có	CSES “Coin Piles”-loại[1]	Sắp tăng dần và duy trì tổng đạt được; bất biến rất gọn.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Mã Huffman	Bài kinh điển	Cài bằng heap; sau đó tự chứng minh bằng đổi chỗ hai ký hiệu hiếm nhất.
•• Cây khung nhỏ nhất bằng Kruskal và bằng Prim	CSES “Road Reparation”[1]	Cài cả hai, so thời gian trên đồ thị thưa và dày; giải thích khác biệt.
•• Xếp lịch giảm thiểu độ trễ lớn nhất	Bài kinh điển	Sắp theo hạn chót; chứng minh bằng đổi chỗ hai việc liền kề.
•• Chọn nhiều việc nhất khi mỗi việc có thời lượng và hạn chót	Bài kinh điển	Tham lam có hồi tiếc với heap; ví dụ mẫu cho mẫu “nhận hết rồi loại”.
••• Sắp thứ tự công việc trên hai máy để tổng thời gian nhỏ nhất	Bài kinh điển	Tiêu chí sắp xếp lại; hãy dùng mẹo đổi chỗ hai phần tử liền kề để tự dẫn ra.
••• Phủ tập hợp bằng tham lam, so với tối ưu trên dữ liệu nhỏ	Bài tập thực nghiệm	Đo tỉ số thực tế so với cận $\ln n$; đây là cách tốt nhất để hiểu “bảo đảm xấp xỉ” nghĩa là gì.
••• Chọn k vị trí đặt cảm biến để phủ nhiều nhất	Bài tập ứng dụng	Hàm mục tiêu dưới mô-đun; tham lam có bảo đảm $1 - 1/e$ — kiểm chứng bằng thực nghiệm.
••• Tìm một hệ mệnh giá làm tham lam đổi tiền sai, càng nhỏ càng tốt	Bài tập thiết kế	Cách rẻ nhất là stress test với vết cạn; bài này luyện đúng thói quen chương E/25.
••• Ghép đôi ổn định	Bài kinh điển	Cài thuật toán cầu hôn; thử chứng minh không tồn tại cặp muốn bỏ nhau — một chứng minh khó hơn vẻ ngoài của thuật toán.

Kết chương và đọc tiếp

Tham lam là chiến lược ngắn nhất về code và đắt nhất về nghĩa vụ chứng minh. Ba điều đáng mang theo: phát biểu chính xác về lý do tham lam sai — phá huỷ khả năng tương lai; mẹo đổi chỗ hai phần tử liền kề để *suy ra* tiêu chí sắp xếp thay vì đoán; và ý thức rằng có một lớp cấu trúc (matroid) mà trên đó tham lam chắc chắn đúng.

Khi không lựa chọn cục bộ nào an toàn, ta buộc phải xét nhiều khả năng — nhưng không phải xét lại từ đầu mỗi lần. Chương tiếp theo là chương dài nhất và quan trọng nhất của phần C: quy hoạch động, kỹ thuật biến “xét mọi khả năng” từ hàm mũ thành đa thức bằng cách nhận ra rằng rất nhiều nhánh khác nhau dẫn tới cùng một trạng thái (C/16).

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).